

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 04 -

VERSIONAMENTO E DISTRIBUIÇÃO INTERN

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	6
MERCADO, CASES E TENDÊNCIAS	20
O QUE VOCÊ VIU NESTA AULA?	24
REFERÊNCIAS.....	26

EMANIP

O QUE VEM POR AÍ?

Imagine a seguinte situação: você atualiza uma biblioteca interna essencial no seu projeto e, de repente, metade do sistema quebra. Situações assim já aconteceram com muitos desenvolvedores e desenvolvedoras — é o temido “inferno das dependências”, em que pequenas mudanças em versões de pacotes geram efeitos cascata inesperados.

Nesta aula, vamos explorar como bibliotecas internas e SDKs podem ser desenvolvidos e gerenciados de forma segura, evitando esse caos. Veremos por que implementar um esquema sólido de versionamento (especialmente o semântico) não é frescura, mas sim uma necessidade para comunicar mudanças de forma consistente. Vamos discutir, por exemplo, Semantic Versioning (SemVer) — aquele padrão MAJOR.MINOR.PATCH — e entender quando incrementar cada número para sinalizar correções, novas funcionalidades ou quebras de compatibilidade.

Também aprenderemos a empacotar e distribuir internamente essas bibliotecas. Ou seja, como preparar o projeto (usando arquivos como `setup.py` ou `pyproject.toml`) e gerar pacotes instaláveis (`.whl` ou `.tar.gz`) para publicar em um repositório privado da empresa (um “PyPI interno”). Por fim, discutiremos boas práticas para gerenciar dependências ao longo do tempo: como definir versões mínimas de dependências, planejar depreciações de funções antigas e comunicar breaking changes aos times de forma pró-ativa.

Tudo isso será apresentado de maneira acessível e prática. Fique conosco e prepare-se para assistir às videoaulas, pois nelas vamos demonstrar esses conceitos em ação, ajudando você a evitar surpresas desagradáveis e a manter seus projetos estáveis e evolutivos.

HANDS ON

Nos vídeos desta aula, você verá passo a passo como criar uma biblioteca Python interna, versioná-la corretamente e publicá-la em um repositório privado. Primeiramente, escolhemos o Python por ser uma das linguagens mais populares em ciência de dados e desenvolvimento de ferramentas internas. Nos vídeos, começamos definindo a estrutura básica do projeto e um arquivo de configuração.

Por exemplo, criamos um arquivo `setup.py` com as informações da biblioteca (nome, versão, dependências etc.). Em paralelo, mostramos como utilizar o moderno arquivo `pyproject.toml` para a mesma finalidade, seguindo as recomendações mais atuais do ecossistema Python (PEP 517/518/621). Isso tudo prepara o terreno para empacotar a biblioteca.

Após configurar o projeto, demonstraremos a geração do pacote distribuível e sua publicação interna. Utilizamos o utilitário `build` do Python para criar os artefatos (`.whl` – Wheel e `.tar.gz` – Source Distribution) e então o `twine` para fazer o upload em um index privado (um servidor de pacotes PyPI interno da empresa).

Durante o processo, relembramos a importância de versionar corretamente: no nosso exemplo prático, começamos com a versão 0.1.0 da biblioteca e simulamos uma atualização de patch (corrigindo um bug) evoluindo para 0.1.1 sem quebrar compatibilidade. Você verá também no vídeo como instalar o pacote a partir do repositório interno usando `pip` com um URL privado. A seguir, trazemos alguns trechos de código extraídos dos vídeos:

```
# setup.py – Configuração básica da biblioteca
from setuptools import setup, find_packages

setup(
    name="biblioteca_exemplo",
    version="0.1.0",
    packages=find_packages(),
    install_requires=["numpy>=1.21,<1.24"], # dependência com versão mínima
    e máxima
```

Código-fonte 1 – Arquivo `setup.py` simplificado para empacotar uma biblioteca interna

Fonte: Elaborado pelo autor (2026)

```
# Gerar o pacote e publicar no repositório interno (PyPI privado)
$ python -m pip install --upgrade build twine
$ python -m build # Gera arquivos .whl e .tar.gz em dist/
$ python -m twine upload --repository-url https://repo.interno/ dist/
```

Código-fonte 2 – Comandos para construir o pacote e enviá-lo ao índice PyPI privado da organização
Fonte: Elaborado pelo autor (2026)

No exemplo, o arquivo `setup.py` define a versão inicial 0.1.0 e especifica que esta biblioteca requer, por exemplo, `numpy` entre as versões 1.21 e 1.23.* (ou seja, `>=1.21,<1.24`). Em seguida, construímos o pacote e o disponibilizamos internamente. Assista às videoaulas para acompanhar todos esses passos em detalhe e ver dicas extras — como onde configurar credenciais no `twine` e como organizar um `pyproject.toml` equivalente. Depois disso, volte aqui e continue para aprofundar os conceitos por trás do que fizemos.

SAIBA MAIS

Agora que você viu a prática, vamos nos aprofundar teoricamente nos pilares deste assunto: versionamento semântico, empacotamento/distribuição interna e gestão de dependências e compatibilidade em bibliotecas internas. Prepare-se para uma discussão técnica densa e rica, conectando esses conceitos com fundamentos de ciência da computação, melhores práticas de engenharia de software e casos matemáticos onde couber. Vamos por partes.

Por que versionamento? O inferno das dependências e o SemVer

Em projetos de software com muitas bibliotecas, é comum enfrentarmos o “dependency hell” – aquela situação em que atualizar um pacote vira um pesadelo porque cada dependência tem requisitos de versões conflitantes. Dois cenários extremos ilustram isso: se as dependências de um projeto estão muito amarradas a versões exatas, você pode ficar incapaz de atualizar uma biblioteca sem ter que atualizar em sincronia todas as outras (o chamado “version lock”).

Por outro lado, se as dependências são definidas de forma muito permissiva (ex.: aceitar “qualquer versão futura”), você pode ser vítima da “promiscuidade de versão”, instalando atualizações que quebram seu código. Ambas as situações levam ao mesmo lugar indesejado: um impasse em que avançar ou não avançar de versão traz riscos enormes.

Para solucionar esse problema, surgiram convenções formais de versionamento — a principal delas é o Versionamento Semântico (Semantic Versioning). A ideia central do SemVer é simples e poderosa: usar um número de versão X.Y.Z (Major.Minor.Patch) em que cada componente transmite a natureza das mudanças realizadas. Resumindo: MAJOR é incrementado para mudanças incompatíveis na API (ou seja, breaking changes), MINOR para adição de funcionalidades mantendo compatibilidade e PATCH para correções de bugs ou ajustes internos que não afetam a compatibilidade.

Isso fornece uma linguagem universal para desenvolvedores(as) e usuários entenderem o que esperar de uma atualização. Por exemplo, se seu projeto vai da

versão 2.3.4 para 3.0.0, já sabemos que houve alterações significativas e potencialmente quebra de compatibilidade; se vai de 2.3.4 para 2.4.0, entendemos que novas features foram adicionadas de forma retrocompatível; e se vai para 2.3.5, provavelmente apenas bugs foram corrigidos.

No contexto de bibliotecas internas, seguir o SemVer ajuda times diferentes a coordenarem integrações. Comunicar mudanças através do número de versão evita surpresas: consumidores de sua biblioteca saberão que uma versão 1.5.0 -> 1.6.0 é segura de atualizar, mas pensarão duas vezes (e lerão as notas de release) antes de subir da 1.x para 2.0.0. Vale mencionar que a especificação oficial do SemVer 2.0.0 (que é adotada amplamente) traz também conceitos de pré-lançamentos (versões alfa, beta, RC) e build metadata, mas esses elementos não influenciam compatibilidade — servem para indicar fases de testes e informações adicionais.

Por exemplo, $1.0.0\text{-alpha} < 1.0.0\text{-beta} < 1.0.0$ (versão final) na ordem de precedência. Observação: em ambientes Python, o PEP 440 define as normas de versionamento de forma compatível com SemVer, incluindo notações como $1.0.0\text{.dev1}$, $1.0.0a1$ (alpha) etc., o que é suportado pelo pip. Assim, uma biblioteca interna em desenvolvimento inicial pode usar $0.1.0\text{.dev2}$ para indicar “versão 0.1.0 em desenvolvimento, build 2”, por exemplo.

Do ponto de vista computacional, um esquema de versionamento formal nos permite até pensar em algoritmos de resolução de dependências. Gerenciadores de pacotes (como pip, conda, Maven, npm etc.) precisam resolver qual conjunto de versões satisfaz todas as restrições dos seus pacotes dependentes – isso é essencialmente um problema de busca/otimização em um grafo de dependências, potencialmente NP-difícil (no caso geral) de resolver. Ferramentas modernas implementam backtracking inteligente e caches para encontrar uma solução viável de versões que não entrem em conflito.

Por exemplo, se o pacote A requer “BibliotecaInterna $\geq 1.0, < 2.0$ ” e um pacote B requer “BibliotecaInterna $\geq 1.5, < 1.8$ ”, o resolvidor vai buscar a versão mais recente de BibliotecaInterna que atenda aos dois (digamos 1.7.z) e instalá-la. Tudo isso só funciona bem se as versões seguirem uma lógica consistente. O SemVer oferece exatamente isso: uma relação de ordem parcial confiável entre versões, de forma que 1.7.5 seja considerado compatível com um pedido de $\geq 1.5, < 1.8$ enquanto 2.0.0 seria rejeitado. Matemática simples de comparação lexicográfica dos componentes

numéricos resolve grande parte do problema, algo trivial para o computador, mas extremamente útil para humanos decidirem ranges (“intervalos aceitáveis”) de versões.

Um insight importante: versão != quantidade de mudanças. Versão indica impacto das mudanças. Uma alteração de 10 arquivos que não quebra nada continua merecendo um incremento MINOR talvez, não MAJOR. Por outro lado, uma única mudança que quebra compatibilidade deve forçar sim um Major bump. Essa disciplina impõe uma reflexão de custo-benefício a cada alteração incompatível: vale a pena causar potenciais ajustes em todos os usuários agora? Muitas vezes, opta-se por adiar uma quebra ou encontrar uma forma compatível de implementar a novidade.

Por isso, projetos maduros evitam sair incrementando major a toda hora – a Microsoft, por exemplo, historicamente manteve certas APIs do .NET Framework estáveis por anos para não obrigar os consumidores a refatorarem código legado. Em contrapartida, quando a mudança é realmente necessária (ex.: melhorias arquiteturais), é saudável assumir o Major release e comunicar claramente. Em resumo, versionamento semântico bem aplicado é parte fundamental da engenharia de software responsável, reduzindo surpresas e alinhando expectativas sobre compatibilidade.

Dica prática: documente e anuncie!

Não basta mudar o número — acompanhe versões com um CHANGELOG (histórico de mudanças) ou notas de release. Liste funcionalidades novas, correções e quebras de API. Assim, mesmo que o SemVer indique o tipo de mudança, os desenvolvedores e desenvolvedoras saberão exatamente o que mudou. Essa comunicação transparente constrói confiança e agiliza a adoção de novas versões. Ferramentas automáticas como Conventional Commits (commits anotados com feat, fix, BREAKING CHANGE etc.) podem ajudar a gerar changelogs e até a sugerir bumps de versão automáticos (há setups de CI/CD que calculam a próxima versão com base nos commits).

Mas mesmo com automação, revisar e pensar no impacto das mudanças continua imprescindível. Como destacado por Hammes (2025), evite erros comuns como: ignorar mudanças públicas (não subir Major quando deveria), ficar eternamente

em versão 0.x (passando instabilidade) ou versionar “pelo número de commits” em vez de pelo impacto na API.

Uma vez que estamos versionando nosso SDK internamente, precisamos entregá-lo aos times que irão usá-lo. Aqui entram técnicas de empacotamento e distribuição interna. Diferentemente de um simples copiar-e-colar de código, empacotar uma biblioteca significa transformá-la em um artefato instalável (por exemplo, um arquivo wheel do Python, ou um JAR no mundo Java, um gem no Ruby etc.), que possa ser facilmente distribuído e gerenciado via um gerenciador de pacotes.

Isso traz vários benefícios de engenharia de software: instalação padronizada com um comando (ex.: `pip install biblioteca_exemplo`), gestão explícita de versões (você instala a versão desejada), isolamento de dependências e possibilidade de rollback (voltar a uma versão anterior se necessário). Em resumo, pacotes versionados são unidades de despliegue reutilizáveis e confiáveis.

No caso do Python, um projeto de biblioteca é estruturado com alguns arquivos de configuração especiais. Historicamente utilizava-se o `setup.py` (como vimos no hands on) – um script Python que usa `setuptools.setup()` para definir metadados do pacote (nome, versão, autor, dependências requeridas etc.). Junto dele, muitas vezes havia um `setup.cfg` (para configurar opções de forma declarativa) e o arquivo `MANIFEST.in` (listando arquivos extras a incluir no pacote).

Entretanto, a tendência moderna é migrar para o `pyproject.toml`, introduzido pelas PEP 517 e 518, e reforçado pela PEP 621, que permite declarar toda a configuração de construção do pacote em formato TOML, de maneira mais padronizada. Dentro do `pyproject.toml`, temos geralmente uma sessão `[build-system]` (indicando qual backend usar para construir – e.g. `setuptools`, `hatch`, `poetry` – e as dependências de build como `wheel`) e a sessão `[project]` onde colocamos os mesmos metadados que antes iam no `setup.py`: nome, versão, descrição, autores, requerimentos de Python e dependências. Por exemplo, poderíamos ter:

```
[build-system]
requires = ["setuptools>=65", "wheel"]
build-backend = "setuptools.build_meta"

[project]
```

```
name = "biblioteca_exemplo"
version = "1.2.0"
description = "Biblioteca interna de exemplo"
readme = "README.md"
authors = [
    { name = "Fulano de Tal", email = "fulano@empresa.com" }
]
dependencies = [
    "numpy>=1.21,<1.24",
    "pandas>=1.3,<1.4" "pandas>=1.3,<1.4"
```

Código-fonte 3 – Sessão [build-system] e sessão [project]

Fonte: Elaborado pelo autor (2026)

Esse `pyproject.toml` usaria `setuptools` como backend mas de forma declarativa, sem código executável. A vantagem dessa abordagem é padronização e isolamento: quando rodamos `pip install` ou `pip build`, o `pip` cria um ambiente isolado, lê o `pyproject.toml` e sabe exatamente como construir o pacote. O processo fica menos sujeito a erros e surpresas e ainda permite facilmente trocar o backend (poderíamos migrar para Poetry ou Hatch sem alterar o workflow do usuário final, só alterando o [build-system]). A partir do `pip 23.1` (2023), o suporte a `pyproject.toml` está maduro e esse é o caminho preferencial para novos projetos. Mas vale notar: o conteúdo final do pacote é o mesmo, usando `setup.py` ou `pyproject` – muitos projetos inclusive mantêm ambos por compatibilidade.

Após escrever nossa configuração, o passo seguinte é construir o pacote. Em Python, usamos a ferramenta de build do PyPA (Python Packaging Authority): `python -m build`. Ela gera dois tipos de artefatos na pasta `dist/`: um arquivo `.tar.gz` (source distribution, contendo fontes) e um `.whl` (wheel, distribuição binária já “montada” para instalação rápida). Se a biblioteca inclui código nativo (C/C++), o wheel será específico para OS/arquitetura; de outro modo, pode ser universal. Esses pacotes podem então ser publicados.

Aqui entra a questão da distribuição interna: em vez de publicar no PyPI público (`pypi.org`), que é aberto a todos, as empresas mantêm seus próprios servidores de pacotes privados para bibliotecas internas ou forks customizados de pacotes externos. Ferramentas populares para isso incluem DevPi, Artifactory (JFrog), Nexus Repository, ou mesmo soluções simples usando um servidor web com diretórios (conforme o protocolo simples do PyPI).

Por exemplo, o Python Packaging Guide mostra que você pode simplesmente organizar uma estrutura de pastas com os nomes dos pacotes e colocar os arquivos .whl/.tar.gz correspondentes, servindo isso via HTTP(S) com autoindex, e pronto: tem-se um índice simples. Claro, softwares dedicados como devpi oferecem recursos extras (cache do PyPI oficial, autenticação, buscas). O importante é que, tendo um índice interno em <https://pypi.empresa.com/simple/>, você consegue instruir o pip a usá-lo. Dessa forma, quando um(a) desenvolvedor(a) quiser instalar a biblioteca_exemplo, ele rodará algo como:

```
pip install --extra-index-url https://pypi.empresa.com/simple biblioteca
```

Código-fonte 4 – Instalando a biblioteca_exemplo

Fonte: Elaborado pelo autor (2026)

O parâmetro --extra-index-url diz ao pip para consultar também o repositório interno além do oficial. Em cenários corporativos, muitas vezes configura-se o pip ou o Poetry de modo permanente para sempre olhar primeiro para o índice privado; algumas empresas até bloqueiam acesso externo, forçando todos os pacotes a passarem pelo índice corporativo (que pode ser um proxy do PyPI público + repositório de privados). Isso traz controle e segurança.

Um dos maiores exemplos da importância de um repositório privado é a prevenção de desastres como o left-pad: em 2016, uma diminuta biblioteca de 11 linhas chamada left-pad foi removida do npm (repositório público JavaScript) pelo seu autor e milhares de projetos pelo mundo quebraram instantaneamente, inclusive aplicações no Facebook, Netflix e outras. A lição: quem tinha copiado ou internalizado o pacote não foi afetado; quem dependia da fonte pública ficou na mão até o npm intervir e restaurar o pacote. Hoje, remover uma versão do PyPI ou npm não é mais permitido facilmente (no PyPI, versões podem ser “yankadas”, mas não deletadas se tiverem mais de 24h). Ainda assim, contar com um mirror interno evita dependência direta de serviços externos e permite controle de qualidade.

Uma vez publicado o pacote no servidor interno, distribuí-lo é questão de segundos: qualquer desenvolvedor(a) da empresa pode instalar a versão específica ou atualizar para a versão mais recente com um simples comando. Isso nos leva a outra prática: integração contínua e artefatos reprodutíveis. Uma boa política é que cada build principal/commit importante da biblioteca gere um pacote (talvez marcado como pré-lançamento) e publique no repositório interno de testes. Assim, times

interessados podem ir validando versões de desenvolvimento (ex.: 1.3.0-dev20251120) antes do lançamento oficial 1.3.0.

Para isso, a automação do CI/CD deve ser configurada e frequentemente atribui-se tags no controle de versão Git para marcar releases (ex.: tag v1.2.0 ligada ao commit exato daquela versão). Ferramentas como GitLab CI, GitHub Actions, Azure DevOps Pipelines etc. podem automatizar o pacote e push no índice interno sempre que uma tag de release for criada, por exemplo. Isso garante que o pacote distribuído corresponda exatamente ao código fonte marcado, permitindo auditoria e debugging preciso.

Do ponto de vista de ciência da computação aplicada, empacotar bibliotecas envolve conceitos de serialização e formatação de código: um arquivo wheel nada mais é que um pacote ZIP com arquivos .py compilados (.pyc) e metadados em um formato bem definido (dist-info). O pip resolver faz parsing desses metadados (pega o METADATA que inclui as dependências requeridas, versão etc.) e assim consegue calcular o grafo de instalação. Toda essa padronização é fruto de PEPs e especificações – por exemplo, a comparação de versões segue algoritmos fornecidos pela própria biblioteca packaging do Python (implementando PEP 440). Para nós, enquanto engenheiros, o proveito é uma implantação mais robusta e escalável: fica muito mais fácil distribuir uma aplicação com dezenas de dependências se todas elas estão empacotadas e versionadas claramente do que se tivéssemos cópias manuais de código espalhadas.

Um detalhe importante é que empacotar também permite gerenciar recursos e performance de forma controlada. Por exemplo, se nossa biblioteca interna faz muito uso de cálculos numéricos pesados, podemos optar por compilar partes críticas em C ou Cython para ganho de velocidade. Nesse caso, ao empacotar, geraríamos wheels específicos para cada sistema (Windows, Linux etc. com suas arquiteturas). O sistema de versionamento de wheel inclui tags para plataforma (como manylinux2014_x86_64), garantindo que um ambiente só instale o wheel compatível. Isso é controle de recursos: podemos ter até variações de implementação trocadas por macros de compilação conforme o ambiente (ex.: usar instruções AVX2 se disponíveis).

Tudo isso é transparente ao usuário da biblioteca – ele só vê a versão e instala. Em certos casos de bibliotecas internas de alto desempenho, também se avalia a

distribuição de processamento: por exemplo, oferecer integração com GPU (possivelmente via dependência opcional). Nesse caso, poderíamos ter uma dependência extra definida no setup (como `extras_require={"gpu": ["cupy>=X.Y"]}`) e o usuário optaria por instalar `pip install biblioteca_exemplo[gpu]`. Enfim, o empacotamento nos dá a flexibilidade de modular e otimizar a entrega do software sem alterar a forma de consumo (sempre via installer).

Outra faceta de performance é a gestão de memória: versões novas de biblioteca podem melhorar o uso de memória e CPU. Ao instruir times internos a atualizarem para versão X ou Y, convém comunicar esses ganhos. Por exemplo, “Na versão 2.0.0 refatoramos o algoritmo de processamento de imagem, reduzindo o uso de memória em ~30% e aproveitando múltiplos núcleos de CPU”. Esse tipo de informação muitas vezes aparece em notas de release de bibliotecas (internas ou open source) e ajuda equipes a decidirem o melhor momento de migrar. Fica a dica: inclua métricas de desempenho nas suas comunicações de release internas sempre que relevante – afinal, para cientistas de dados e engenheiros, saber que o novo SDK roda uma análise em 10 minutos em vez de 30 é um incentivo e tanto para atualizar!

Boas práticas de dependências e compatibilidade a longo prazo

Chegamos ao ponto crucial de manter uma biblioteca interna útil e confiável ao longo do tempo. Seguir SemVer e empacotar são passos fundamentais, mas precisamos ativamente gerenciar dependências e compatibilidade. Aqui estão algumas melhores práticas consolidadas:

a) Defina intervalos de versões de dependências com critério: quando sua biblioteca interna depende de outras (por exemplo, uma biblioteca de visualização interna que usa matplotlib), especifique no setup/pyproject a versão mínima necessária daquela dependência e eventualmente um limite máximo se você souber que versões futuras podem quebrar. Por exemplo, `matplotlib>=3.5,<4.0`. Essa especificação evita instalar sem querer uma versão muito antiga (que faltaria funcionalidade) ou uma major release futura possivelmente incompatível.

Essa prática é exemplificada no próprio guia do SemVer: se seu projeto “CaminhãoBombeiros” foi construído com a lib “Escada” versão 3.1.0, você pode declarar dependência `>=3.1.0, <4.0.0` com segurança. Assim, quando “Escada” lançar

3.2.0 ou 3.1.1, você pode aceitá-las sem medo, mas se um dia sair a 4.0, seu projeto não irá usá-la automaticamente – evitando quebra inesperada. Essa estratégia elimina a “promiscuidade” e o “lock” simultaneamente: dá flexibilidade dentro de um major estável. Leve essa ideia para suas bibliotecas internas e também para aplicações que as consomem: no arquivo de requisitos de um serviço, por exemplo, pinte ranges compatíveis em vez de “pegar sempre o mais recente” ou “ficar congelado em uma versão só para sempre”.

b) Tenha um plano de depreciação de funcionalidades: conforme sua biblioteca evolui, algumas funções/classes podem ficar obsoletas – talvez porque uma abordagem melhor surgiu ou porque regras de negócio mudaram. Em vez de simplesmente removê-las de supetão (o que causa breaking change imediato), use um ciclo de vida de depreciação. Ou seja, marque como depreciado em uma versão MINOR e só remova de fato na próxima MAJOR. Marcar como depreciado pode incluir: adicionar um decorador ou aviso (no código Python, usar o módulo warnings para emitir um DeprecationWarning quando a função é usada), atualizar a documentação indicando “esta função será removida na versão 2.0, use XYZ em seu lugar” e comunicar nas notas de versão. Isso dá aos times consumidores tempo para se adaptar.

Uma boa prática é oferecer, sempre que possível, alternativas para o que está sendo depreciado. Exemplo: “A função `calcula_metricas(df)` está obsoleta; favor usar `calcular_metricas(df, metodo="padrao")`. Ela será removida na versão 3.0.” – junto disso, prove um guia de migração se necessário. Organizações maiores formalizam isso: o time do Fluid Framework da Microsoft, por exemplo, categorizava notas de mudança em “Breaking Changes” (para remoções efetivas em majors) e “Upcoming Changes” (avisos prévios em minors indicando futuras quebras). Seguir esse tipo de abordagem fortalece a confiança dos usuários: eles não descobrirão na marra que algo sumiu, pois já foram avisados com antecedência e instruídos sobre a nova forma de fazer.

c) Breaking change? Só em major – mas documente mesmo assim: segundo o SemVer, breaking changes devem obrigar o aumento da versão major. Isso é regra de ouro. Então nunca introduza uma mudança incompatível em uma atualização minor ou patch – resista à tentação. Se acontecer por acidente (afinal, errar é humano), corrija assim que possível lançando uma versão corretiva e trate

aquela versão como problemática. Ao lançar um major que quebra compatibilidade, explique o porquê e os benefícios disso. Pode parecer excesso de zelo, mas comunicar a razão de uma quebra (“refatoramos para melhorar a segurança de autenticação”, “trocamos a biblioteca X pela Y para ter melhor performance”) faz os usuários aceitarem melhor o esforço de atualizar. Eles verão valor na mudança em vez de apenas trabalho. E claro, ofereça guias de migração: se possível, até scripts de conversão ou linters que apontem o uso das APIs antigas.

Isso foi fundamental, por exemplo, na transição do Python 2 para 3 – embora tenha sido fora do SemVer (o Python não seguia SemVer naquela mudança), a comunidade criou ferramentas como 2to3 para auxiliar. No caso de nossas bibliotecas internas, podemos adicionar testes de compatibilidade ou pacotes shim durante um período (ex.: manter uma função antiga chamando a nova internamente, apenas emitindo alerta, durante algumas versões). Novamente, o time Fluid da Microsoft recomenda em suas diretrizes internas: se algo foi marcado obsoleto na 1.x e removido na 2.0, certifique-se de que a 2.0 tenha nas release notes o lembrete dessa remoção e como ajustar – nunca é demais reforçar.

Outra técnica é usar versionamento de namespace ou pacote em casos extremos: por exemplo, o Google Cloud às vezes lança bibliotecas v1 e v2 coexistindo (módulos separados) quando a mudança é enorme; mas isso é mais comum em SDKs externos. Internamente, é preferível evoluir no mesmo pacote, mantendo controle via versão.

d) Mantenha compatibilidade retroativa sempre que viável: extensão do item anterior – procure adicionar recursos de forma compatível em vez de alterar o comportamento de recursos existentes. Se precisar mudar o comportamento de algo (ex.: um cálculo que antes retornava um valor arredondado e agora precisa retornar exato), veja se dá para introduzir via uma opção ou nova função, depreciando a antiga, em vez de simplesmente mudar silenciosamente. Assim, consumidores podem adotar a novidade gradualmente. Lembre-se: se você providencia uma API estável e clara, respeite esse contrato. É como uma promessa: outros sistemas confiam nisso. Quebrar compatibilidade pode ter um impacto grande (um sistema em produção pode falhar).

Essa preocupação é tão grande que alguns sistemas adotam testes de compatibilidade: rodar a bateria de testes dos projetos dependentes contra a nova

versão da biblioteca. Em uma escala de comunidade open source, isso nem sempre é possível, mas internamente é factível se você conhece seus “clientes”. Por exemplo, antes de lançar BibliotecaInterna 2.0, você poderia testar localmente os serviços A, B e C da empresa que usam a versão 1.x para ver se tudo passa, e talvez já abrir pull requests de ajuste nesses serviços antecipadamente. Isso é colaboração intequipes e economiza dor depois do release.

e) Gerencie as dependências da sua biblioteca como um produto vivo:

isso significa acompanhar atualizações dos pacotes de terceiros que sua biblioteca usa. Se você fixou `numpy<2.0` por precaução, quando o NumPy 2.0 eventualmente sair, você vai querer testar sua biblioteca com ele, possivelmente adaptar algo e então lançar uma versão nova removendo a restrição (ou elevando para `<3.0`). Não deixe suas dependências defasarem sem triagem – isso leva ao acúmulo de dívida técnica. Um indicador matemático: se você pula muitas versões de um pacote, a probabilidade de dificuldades ao atualizar cresce. Atualizações pequenas frequentes tendem a ser menos custosas do que pular direto após anos.

Ferramentas como Dependabot podem automatizar notificações de novas versões de dependências no seu repositório interno. Leve em conta também atualizações de segurança: se uma dependência lança um patch crítico (ex.: corrigindo falha de segurança), incorpore-o na sua biblioteca o quanto antes (lançando um patch release seu). Um exemplo famoso foi a vulnerabilidade Log4Shell no Log4j (mundo Java) em 2021: inúmeros sistemas internos corporativos precisaram atualizar do Log4j 2.14 para 2.16 ou superior às pressas. Quem mantinha bibliotecas internas envolvendo logging teve que soltar novas versões e comunicar os times consumidores imediatamente. Portanto, esteja atento(a) a boletins de segurança das suas dependências e tenha um plano de resposta rápida.

f) Controle de qualidade contínuo: testes automatizados (unitários, integrados) na sua biblioteca garantem que mudanças de implementação não quebrem contratos esperados. Inclua testes para comportamentos legados também, principalmente ao marcar algo como depreciado (para garantir que enquanto existir, funciona). Se possível, monitore o uso da biblioteca: por ser interna, você pode ter métricas de telemetria ou feedback dos times. Por exemplo, se quase ninguém usa a função X, talvez removê-la não seja tão traumático – mas se for algo crítico, redobre a cautela.

g) Considere versionamento em diferentes dimensões: em casos complexos, às vezes a compatibilidade não é apenas função da versão do código, mas do ambiente. Exemplo: uma biblioteca interna que interage com um serviço remoto pode ter que sincronizar versões com o contrato desse serviço. Nesses casos, deixe claro o relacionamento (ex.: “Biblioteca Cliente v2.x é compatível com Servidor API v1, já a v3.x funciona com Servidor API v2”). Esse tipo de mapeamento pode virar uma tabela em documentação ou mesmo ser codificado: algumas SDKs incluem no user-agent ou em constantes a versão de API alvo.

Assim, ao evoluir o servidor, você poderia manter compatibilidade com N versões antigas do client. Esse gerenciamento de compatibilidade cross-sistema é parte da arquitetura geral, mas convém mencionar aqui para lembrar: versionamento semântico foca na API da própria biblioteca, mas quando ela faz parte de um ecossistema maior, planeje as evoluções de forma coordenada. É frustrante para um time atualizar a biblioteca e descobrir que agora precisam atualizar o serviço backend junto — se isso não estiver alinhado, gera downtime. Logo, major releases de SDKs internos frequentemente acompanham milestones de produto.

Para resumir essas práticas de forma concreta, vamos simular um pequeno ciclo de vida de mudanças bem gerenciado: suponha que nossa biblioteca_exemplo v1.2.0 expõe a função `analisa_dados(df)` mas vamos substituí-la por `analisar_dados(df, algoritmo="novo")` que tem comportamento melhor. Na versão 1.3.0 (MINOR) lançamos a nova função e marcamos `analisa_dados` como depreciada, emitindo warning quando usada. Comunicamos: “Função `analisa_dados` será removida na 2.0; use `analisar_dados`.” Os times ajustam-se nos meses seguintes. Quando estivermos prontos para a 2.0.0 (MAJOR), removemos `analisa_dados` do código completamente e incrementamos o major. Como todos já estavam cientes e migrando, o impacto é mínimo. Essa timeline de depreciação planejada poupa muitas madrugadas de correção urgente depois.

Conexão com projetos de Ciência de Dados e desempenho

Como este curso é de pós-graduação em ciência de dados e engenharia, vale fazer algumas conexões específicas com esse domínio. Pense em projetos de data science em sua organização: frequentemente, equipes de DS criam seus próprios

pacotes utilitários internos – por exemplo, uma biblioteca de feature engineering, ou um SDK para acessar dados corporativos, ou um conjunto de métricas de avaliação de modelos. Tais bibliotecas são fundamentais para padronizar e acelerar o trabalho e por isso precisam ser versionadas e distribuídas tal qual discutimos.

Um caso prático: imagine uma biblioteca interna de feature store que várias squads usam para acessar e transformar dados. Se ela não for bem versionada, um ajuste em uma transformação poderia alterar discretamente os resultados de modelos historicamente treinados, causando diferença entre offline (treino) e online (inferência). Para evitar isso, as empresas mantêm versões estáveis para pipelines de ML: um modelo em produção pode continuar usando a versão 1.x da lib de features até que se valide que atualizar para 2.x não muda drasticamente as entradas do modelo (ou muda para melhor). Ou seja, reprodutibilidade em ciência de dados está intimamente ligada a controle de versões de bibliotecas e ambientes. Em termos estatísticos, garante-se que a distribuição dos dados de entrada não mudou devido a uma atualização de código inadvertida.

Além disso, ao desenvolver bibliotecas internas para DS, questões de performance são críticas: versões novas devem idealmente ser mais rápidas ou mais eficientes que as antigas (a menos que se troque desempenho por acurácia conscientemente). Podemos quantificar melhorias – por exemplo, “versão 1.4 do pacote de recomendações reduziu o tempo médio de predição de 120ms para 80ms (33% mais rápido) graças a vetorização do código”. Essas métricas podem e devem aparecer na comunicação de release.

Se sua atualização major torna algo mais lento, seja transparente sobre o porquê (talvez cálculos mais complexos, ou garantias de exatidão maiores). Em projetos de análise de dados intensivos, o uso de recursos computacionais (CPU, GPU, memória) costuma ser monitorado: então, atualizações de bibliotecas que afetem esses números serão notadas. Faça testes de benchmark entre versões se possível e publique no Saiba Mais da sua documentação técnica interna. Isso alinha expectativas: “olha, trocamos o algoritmo X pelo Y, ficou 10% mais lento em datasets pequenos mas escala melhor para datasets grandes, consumindo 20% menos memória – faz sentido dependendo do caso de uso”.

Também não esqueçamos da distribuição paralela: se você introduzir uso de threads ou processamento distribuído (Spark, Dask etc.) em uma biblioteca interna,

avaliar muito bem a compatibilidade. Exemplo: sua função antes era single-thread e agora passa a usar todos os núcleos – isso pode ser ótimo para velocidade, mas também pode impactar servidores compartilhados ou consumir todo recurso de máquina do usuário.

Documente essas mudanças e, se aplicável, forneça flags para controlar (e.g., número de threads configurável). Em termos de computação, você está alterando o perfil de uso de CPU e possivelmente condições de corrida podem surgir – aumente a suíte de testes para cenários concorrentes. Pode parecer detalhe, mas é fundamental lembrar: uma biblioteca interna vive dentro de um ecossistema, então cada alteração pode ter efeitos globais.

Para fechar esta seção, reforçamos: encare suas bibliotecas internas com o mesmo rigor de software profissional que você teria com um produto externo. Isso inclui versionamento semântico disciplinado, empacotamento e entrega confiáveis e um ciclo de vida bem administrado. Ferramentas existem para ajudá-lo (desde linters até sistemas de gestão de artefatos), mas entender os princípios base – como fizemos aqui – vai permitir que você tome decisões informadas quando surgirem dilemas. Agora, vamos ver alguns casos reais de mercado e tendências relacionadas para cimentar ainda mais esses conceitos.

MERCADO, CASES E TENDÊNCIAS

No panorama atual da indústria de software, o gerenciamento de bibliotecas internas e seu versionamento ganhou ainda mais destaque devido a dois fatores: a crescente complexidade das aplicações (microserviços, multiplataforma, data lakes etc.) e as preocupações de segurança da cadeia de suprimentos de software (software supply chain). Vamos analisar algumas tendências e casos.

DevOps e CI/CD impulsionando versionamento automatizado: com a popularização de pipelines de integração contínua, surgiu uma tendência de automatizar o processo de release. Ferramentas como semantic-release (muito usada em projetos Node.js) interpretam mensagens de commit e automaticamente geram uma nova versão seguindo SemVer, publicando no registro. No mundo corporativo, vemos a adoção de fluxos semelhantes: merges marcados como feat ou fix vão acumulando mudanças até que, no momento de release, o pipeline decide entre bump de patch/minor/major.

Isso agiliza entregas, mas requer disciplina nos commits. Empregar automação assim para bibliotecas internas pode reduzir trabalho manual e garantir consistência de versionamento. Entretanto, mesmo a automação exige revisão humana para evitar que um commit mal categorizado gere um major indevido ou esqueça de indicar uma breaking change. Por isso a combinação de Conventional Commits + revisão de código + CI é considerada uma boa prática emergente.

Monorepo vs Multirepo: uma discussão em alta é sobre estratégias de repositório de código. Em monorepos, toda a base de código da organização (ou de um grande produto) fica em um repositório único e ferramentas internas gerenciam builds e dependências sem exigir versionamento formal para cada módulo – o commit é a fonte da verdade. Google e Meta são exemplos famosos de monorepos massivos. Isso evita problemas de incompatibilidade de versões, mas requer engenharia pesada de build.

Já no modelo multirepo, cada biblioteca vive em seu repositório e versiona independentemente (mais comum na maioria das empresas). Vemos algumas empresas híbridas: por exemplo, a Microsoft adota monorepos por produto, mas cheios de pacotes versionados internamente que são consumidos por outros times via

feeds do Azure Artifacts. A tendência atual parece ser: startups iniciam com monorepo (pela facilidade de coordenação inicial), mas conforme escalam e equipes ficam mais independentes, migram para pacotes separados com versionamento (ou usam um monorepo com ferramentas de versionamento interno, como Lerna ou Nx no ecossistema JS, ou repositórios federados).

Para nossos fins, o importante é entender que, mesmo em monorepos, conceitos de SemVer e compatibilidade não desaparecem – eles apenas podem ser aplicados de forma mais “soft”, via tags ou convenções internas. Já em múltiplos repositórios, o que falamos é lei para manter a orquestração do sistema.

Segurança e o movimento “SBOM”: incidentes recentes de segurança elevaram a prioridade de gerenciar bibliotecas (próprias e de terceiros) de forma rastreável. Termos como SBOM (Software Bill of Materials) se referem a listar todas as dependências e suas versões em software entregue, para possibilitar auditorias rápidas. Por exemplo, após a vulnerabilidade Log4Shell, era crítico saber quais sistemas usavam log4j e em que versão. Ferramentas de análise estática e scanners (Snyk, OSS Audit, WhiteSource) vasculham projetos para identificar versões vulneráveis. Para bibliotecas internas, isso implica manter transparência das dependências que elas trazem.

Uma tendência de mercado é incorporar dados de versão no pipeline de deploy – ex.: gerar um SBOM em formato SPDX para cada build, facilitando rastreamento. Organizações governamentais e setores regulados estão começando a exigir isso dos fornecedores. Portanto, se sua biblioteca interna é usada em um produto final, já se espera que ela siga boas práticas de versionamento (SemVer) e agora também que exponha claramente suas dependências de terceiros e licenças. Não se assuste se em um futuro próximo você precisar adicionar ao seu repositório interno um arquivo SPDX ou CycloneDX enumerando “pandas 1.3, numpy 1.21” etc., para compliance.

Casos de sucesso e fracasso no uso de bibliotecas internas: é difícil achar relatos públicos de “fracasso” com bibliotecas internas (afinal, coisas internas raramente viram notícia), mas podemos inferir lições de casos conhecidos. Um caso de sucesso famoso é o da Netflix, que conseguiu escalar sua arquitetura de microsserviços em parte graças a bibliotecas internas bem gerenciadas. Eles criaram o chamado Simian Army (conjunto de ferramentas como Chaos Monkey) e vários

clientes para facilitar autenticação, convenções de REST etc., que eram compartilhados entre os serviços.

Documentos da Netflix Engineering indicam que eles adotaram versionamento semântico e liberavam frequentemente essas libs via repositório Maven privado, garantindo que todos os serviços estivessem atualizados com as melhores práticas de resiliência. Ao mesmo tempo, a Netflix também abraçou o open source – muitas dessas bibliotecas foram abertas, o que exigiu ainda mais rigor de estabilidade para receber contribuições externas.

Por outro lado, um “fracasso” ilustrativo é o de empresas que subestimaram o versionamento: já houve casos (anônimos, relatados em blogs ou conferências) de times que quebraram a produção ao atualizar uma biblioteca interna sem perceber mudanças incompatíveis. Por exemplo, um time de e-commerce quebrou o fluxo de checkout porque uma função de cálculo de imposto em uma lib interna alterou a forma de arredondamento na versão nova – faltou comunicação e teste, resultando em valores de cobrança errados em alguns centavos, um prejuízo reputacional grande. Esse tipo de incidente reforça porque insistimos em depreciar gradualmente e versionar semanticamente: se a função de imposto fosse mantida retrocompatível até um major, e/ou se o time de negócio fosse alertado da mudança no cálculo, o rollout seria controlado.

Evolução das ferramentas de package management: tanto no mundo Python quanto em outras linguagens, as ferramentas estão evoluindo para suportar melhor cenários corporativos. O PyPI introduziu o conceito de yank (mencionado anteriormente): você não apaga uma versão defeituosa, mas a marca como yanked. Assim, quem especificar explicitamente aquela versão ainda consegue instalar (com um aviso), mas quem pede “a mais recente” vai pular ela. Isso evita os erros 404 de dependência e permite corrigir problemas sem quebrar builds alheios.

Ferramentas de instalação (pip, npm) também estão ficando mais inteligentes em resolver conflitos e managers alternativos como poetry e pipenv adotam lock files (Poetry.lock, Pipfile.lock) similar ao package-lock.json do npm para congelar versões exatas quando desejado – útil para aplicações finais em produção, garantindo que em cada deploy as mesmas versões são usadas, apesar de novas versões das libs irem surgindo no repo interno. Em paralelo, tem surgido integração de Git e pacotes: por exemplo, o GitHub Packages permite publicar pacotes diretamente no entorno GitHub

e até usar um repositório Git “puro” como índice (algumas empresas fazem isso para projetos pequenos). O TheLinuxCode mostrou como GitHub/GitLab podem servir como PyPI privado aproveitando permissões já definidas no repositório. Isso é tendência pois unifica gestão de código e artefato em um só lugar – menos infraestrutura para manter.

Calendário e versionamento preditivo: uma outra tendência não tão forte em bibliotecas internas, mas existente em projetos maiores, é usar versionamento por data (CalVer) em vez de SemVer para certos componentes. Por exemplo, o Ubuntu Linux lança versões 20.04, 22.04 (ano.mês) e algumas empresas fazem algo parecido internamente para indicar “release Q4 2025” por exemplo. O Python pip e setuptools usam CalVer (pip 23.2 etc.).

Em bibliotecas internas de data science, poderíamos imaginar uma versão “2023.1” para o pacote de features alinhado com a primeira entrega de 2023 e assim por diante. Isso deixa óbvio quão “atrasado” está um ambiente (se alguém ainda usa 2021.x em 2025, está 4 anos defasado). Entretanto, o CalVer não comunica automaticamente compatibilidade – tendemos a assumir que todo release anual pode ter quebra. Em projetos que seguem CalVer, geralmente adota-se políticas claras de depreciação também (ex.: “suportamos as versões dos últimos 2 anos”). É válido conhecer, mas para SDKs focados em API, SemVer costuma ser suficiente.

Para concluir esta seção: o gerenciamento de bibliotecas internas evoluiu de um aspecto técnico discreto para um elemento estratégico no desenvolvimento de software. Empresas enxergam que dominar isso aumenta a qualidade e segurança de seus produtos. Como tendência futura, podemos esperar mais automatização (e.g. bots abrindo PRs para atualizar libs internas nos projetos dependentes assim que uma nova versão sai, semelhante ao Dependabot para libs externas) e mais observabilidade – talvez painéis internos mostrando quais equipes estão desatualizadas e até pontuação de “saúde” de dependências. Um(a) engenheiro(a) de software hoje precisa transitar entre escrever código eficiente e gerir o impacto desse código em um ecossistema maior.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula abordamos de forma abrangente como criar e manter **bibliotecas internas e SDKs** com qualidade profissional. Como checkpoint de aprendizado, vamos revisar rapidamente os pontos-chave.

Aprendemos o esquema MAJOR.MINOR.PATCH e como ele comunica claramente a natureza das mudanças em uma biblioteca. Vimos que versionamento consistente previne o inferno das dependências e facilita a coordenação entre times, garantindo compatibilidade retroativa e evolução previsível.

Exploramos como preparar uma biblioteca para uso interno, usando arquivos como `setup.py` ou `pyproject.toml` para definir metadados e dependências e gerando pacotes (como wheels) instaláveis via `pip`. Discutimos a configuração de repositórios PyPI privados e como publicar e instalar pacotes neles, assegurando controle de acesso e confiabilidade. Você viu na prática a criação de um pacote Python de exemplo e entendeu que empresas usam ferramentas como DevPi, Nexus ou Artifactory para hospedar suas “lojas” internas de pacotes.

Destacamos estratégias para gerenciar a evolução de uma biblioteca sem quebrar quem a usa: uso de versões mínimas/máximas de dependências de terceiros, uso de avisos de depreciação e APIs alternativas antes de remoções definitivas, respeito ao SemVer (breakings só em major), manutenção de documentação (CHANGELOG, README de upgrade) e colaboração entre times para testar e adotar novas versões. Tudo isso visando reduzir surpresas e construir confiança na sua SDK interna.

Observamos que melhorias (ou regressões) de desempenho entre versões devem ser medidas e comunicadas, pois impactam diretamente usuários e projetos de data science. Também vimos que reprodutibilidade de experimentos e estabilidade de modelos em produção estão ligadas ao versionamento adequado das bibliotecas internas que fornecem dados e funções para esses modelos.

Por fim, analisamos acontecimentos reais (como o incidente left-pad) que enfatizam a importância de boas práticas de versionamento e distribuição. Reconhecemos o movimento das empresas em implementar repositórios internos e automatizar seus pipelines de release, bem como a crescente atenção à segurança

de dependências (SBOM). Isso nos mostrou que o conhecimento desta aula não é apenas teórico: ele responde a necessidades atuais da indústria de software em larga escala.

Com isso, você deve estar apto(a) a planejar e implementar uma biblioteca interna desde a concepção até a manutenção contínua, utilizando versionamento semântico, empacotamento profissional e comunicação clara com sua equipe e demais consumidores. Agora, não deixe de conferir as videoaulas associadas e partir para as atividades práticas propostas – coloque em ação esses conceitos criando um pacote de exemplo e simulando alguns releases. Reforce seu aprendizado explorando também as referências e voltando em pontos do material escrito sempre que precisar. Bom trabalho e bons commits versionados!

REFERÊNCIAS

CRAFTEDSTACK. **CraftedStack (Zachary Carciu) – How to Build and Publish Your Python Package.** 2025. Disponível em: <https://craftedstack.com/blog/python/build-and-publish-python-package/> . Acesso em: 21 jan. 2026.

HAMMES, R. **Melhores Práticas de Versionamento Semântico.** 2025. Disponível em: <https://rodrigohammes.com/posts/melhores-praticas-de-versionamento-semantico-como-aplicar-semver-com-seguranca-nos-seus-projetos> . Acesso em: 21 jan. 2026.

PACKAGING PYTHON. **Guia de Empacotamento de Projetos Python – Python Packaging User Guide.** 2026. Disponível em: <https://packaging.python.org/en/latest/tutorials/packaging-projects/> . Acesso em: 21 jan. 2026.

PACKAGING PYTHON. **Hosting your own simple repository – Python Packaging User Guide.** 2026. Disponível em: <https://packaging.python.org/en/latest/guides/hosting-your-own-index/> . Acesso em: 21 jan. 2026.

SEMVER. **Versionamento Semântico 2.0.0 – semver.org (versão oficial em Português).** 2026. Disponível em: <https://semver.org/lang/pt-BR/> . Acesso em: 21 jan. 2026.

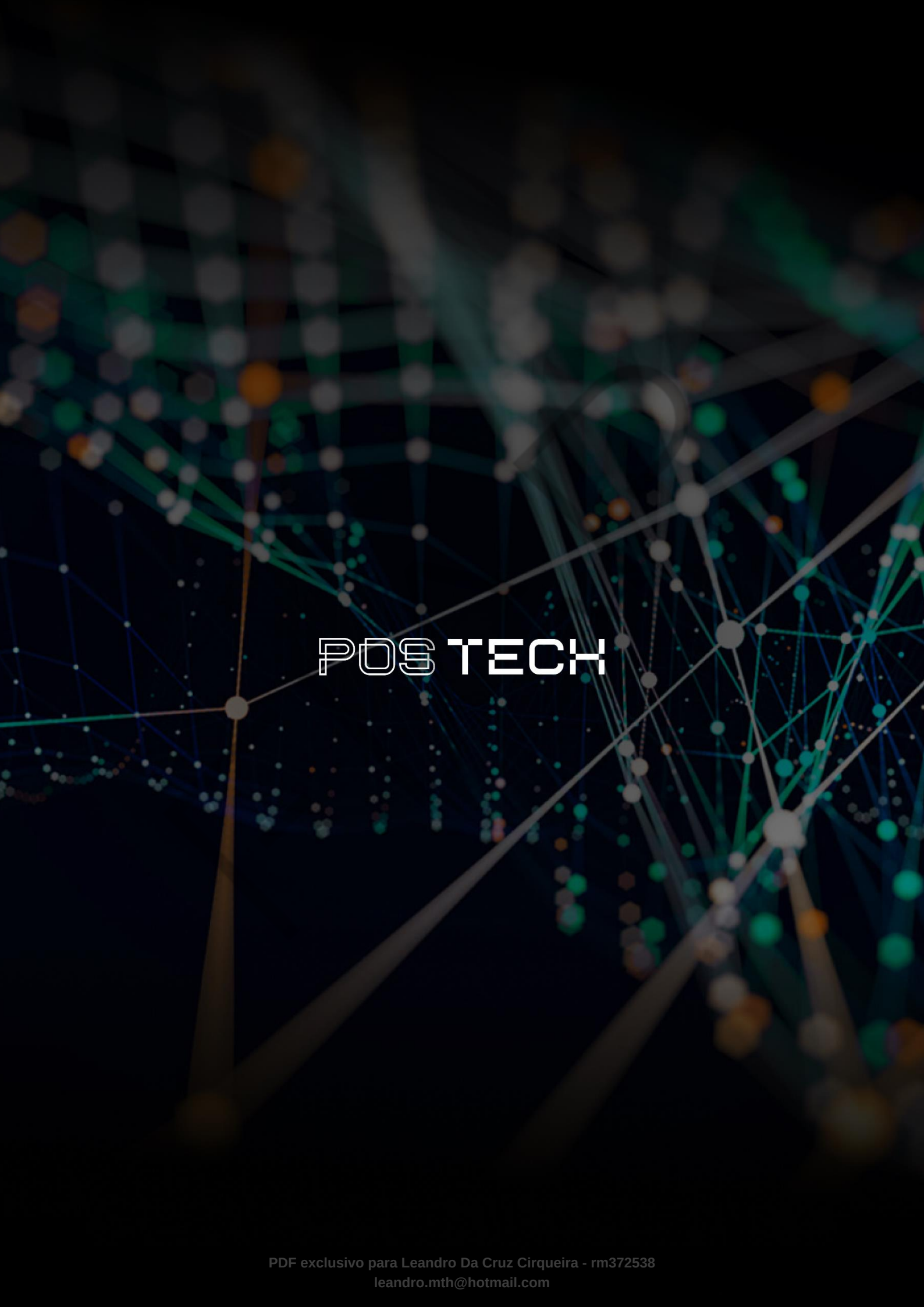
THE LINUX CODE. **TheLinuxCode – How to Use GitHub as a Private PyPI Server.** 2024. Disponível em: <https://thelinuxcode.com/how-to-use-github-as-a-private-pypi-server/> . Acesso em: 21 jan. 2026.

WIKIPEDIA. **Wikipedia – npm left-pad incident.** 2026. Disponível em: https://en.wikipedia.org/wiki/Npm_left-pad_incident . Acesso em: 21 jan. 2026.

PALAVRAS-CHAVE

Versionamento Semântico. Repositório Privado de Pacotes. Compatibilidade Retroativa.

EMSE



POSTECH